

Face & Fingerprint Embedding Training

A step-by-step technical walkthrough of the two Kaggle training notebooks used in Phase 1 of the Cancelable Multimodal Biometric Authentication project — what each cell does, which computer-vision techniques are used and why, how InceptionResnetV1/VGGFace2 and ResNet50 are used as embedding backbones, and an honest read of the real training outcomes.

Source notebooks: [kaggle_kernels/face_training/face-embedding-training.ipynb](#) & [kaggle_kernels/fingerprint_training/fingerprint-embedding-training.ipynb](#)

Supporting reference: [docs/COMPUTER_VISION.md](#), [docs/PROJECT_REPORT.md](#)

Contents

1. Overview — what these notebooks train and why
2. Face Training Notebook — step by step
3. Computer Vision Techniques Used for Face
4. InceptionResnetV1 Architecture, in Detail
5. VGGFace2 Pretraining and Transfer Learning
6. ArcFace: the Shared Fine-Tuning Loss
7. Face — Real Training Outcome
8. Fingerprint Training Notebook — step by step
9. Computer Vision Techniques Used for Fingerprint
10. ResNet50 + Projection Head Embedding
11. Fingerprint — Real Training Outcome
12. Why Fingerprint Underperformed Face (Honest Analysis)
13. Side-by-Side Comparison

1. Overview

Both notebooks fine-tune the embedding model a specific biometric modality uses at inference time (`models/face/inference.py` and `models/fingerprint/inference.py`), then evaluate the result with standard biometric-verification metrics (accuracy at the Equal Error Rate threshold, EER itself, and ROC-AUC) computed identically for every modality by the shared `evaluation/` module. Both run unattended on Kaggle's free-tier GPU, cloning the project repository at the start of the kernel rather than requiring any manual upload.

Every biometric pipeline in this project has exactly two stages, and it matters to be precise about which stage is which:

- **Preprocessing** — takes a raw image and produces a normalized, modality-specific representation. For fingerprint this is *entirely classical computer vision* (no learned weights at all). For face, one small pretrained detector (MTCNN) is used for the classical CV task of finding and aligning the face region.
- **Embedding** — takes the preprocessed representation and produces a fixed-length numeric vector using a deep convolutional neural network, fine-tuned with metric learning (ArcFace). This stage is deep learning for both modalities.

Modality	Preprocessing (classical CV)	Embedding (deep learning)
Face	MTCNN detection + landmark-based geometric alignment	InceptionResnetV1, 512-d, pretrained on VGGFace2
Fingerprint	CLAHE contrast enhancement + statistical ridge normalization + Gabor filter bank enhancement	ResNet50 + projection head, 256-d, pretrained on ImageNet

2. Face Training Notebook — Step by Step

2.1 Setup

The kernel clones the repository at the phase-1-foundation branch (Phase 1 code is not yet merged to main), installs facenet-pytorch and h5py, and checks CUDA. Rather than trusting `torch.cuda.is_available()` alone, it runs a real tensor operation as a smoke test, because some Kaggle GPU sessions report CUDA as available but actually crash with *"no kernel image is available for execution on the device."* If that op fails, the run silently falls back to CPU rather than crashing deep into training.

What actually happened: in the logged run that produced the checkpoint currently in this repository, the smoke test failed — the log shows `CUDA available: True` immediately followed by `Using device: cpu`. This face model was trained on CPU, not GPU.

2.2 Download the dataset

Downloads **LFW** (Labeled Faces in the Wild) via `sklearn.datasets.fetch_lfw_people(min_faces_per_person=20, ...)` — this filters to identities with at least 20 photos, since enough images per person are needed for both training and genuine-pair verification testing later. No login or manual download required.

Real run: **3,023 images across 62 identities**.

2.3 Preprocess (detect + align) every image

Every image is run through `preprocessing/face.py`'s `FacePreprocessor` — the exact code path production inference uses, so training sees identically-processed input. Images where no face is detected are dropped rather than failing the whole run.

Real run: **2,984 aligned, 39 skipped** (no face detected).

2.4 Train / validation / test split

Split 70/15/15 **per identity** (not globally) using a fixed seed (`rng.default_rng(42)`), guaranteeing every split retains multiple images of the same person — required because the evaluation stage needs genuine (same-person) pairs to compute FAR/FRR/EER.

Real run: **train = 2,065, val = 423, test = 496**.

2.5 Model: InceptionResnetV1 backbone + ArcFace head

The backbone is `InceptionResnetV1` pretrained on VGGFace2 (512-d output). An `ArcMarginProduct` head (ArcFace) is attached on top, sized to this run's 62 identities. Only the last Inception block plus the final linear/batch-norm layers are unfrozen:

```
for name, param in backbone.named_parameters():
    param.requires_grad = any(
        name.startswith(p) for p in ['block8', 'last_linear', 'last_bn']
```

)

This is a standard transfer-learning pattern: keep the low- and mid-level filters (edges, textures, local facial-part detectors) that VGGFace2 pretraining already learned well, and only adapt the highest-level, most task-specific layers to this project's own enrolled identities. Sections 4-6 explain this architecture and the ArcFace loss in full depth.

2.6 Training loop

Pixels are normalized as $(x - 127.5) / 128.0$ (facenet-pytorch's expected input convention). Standard supervised loop: forward through backbone → 512-d embedding → ArcFace head → cross-entropy loss, Adam optimizer (lr = 1e-4), batch size 32, 10 epochs.

Epoch	Train Loss	Train Acc	Val Acc
1	13.4818	0.126	0.291
2	8.3261	0.334	0.416
3	5.7416	0.418	0.506
4	3.7445	0.545	0.619
5	2.3631	0.653	0.704
6	1.3983	0.758	0.783
7	0.7622	0.848	0.853
8	0.3657	0.917	0.887
9	0.1804	0.965	0.901
10	0.1378	0.976	0.913

Loss drops steadily and accuracy climbs every epoch, with validation tracking training closely — a clean, non-overfit convergence.

2.7 Save the checkpoint

Only `backbone.state_dict()` is saved — not the ArcFace head, which is training-only scaffolding — to both `face_embedder.pt` (canonical, loaded by production inference code) and `face_embedder.h5` (interoperability export via `models/common/checkpoint_io.py`).

2.8 Evaluation — Experiment 1

The just-saved checkpoint is reloaded through the actual production `FaceEmbedder` class (a real end-to-end check that the weights load correctly at inference). Embeddings are extracted for every test image, then `evaluation/experiments.py`'s `run_modality_experiment`:

- Computes cosine similarity for **every pair** of test embeddings, splitting into genuine (same identity) vs. impostor (different identity) score distributions.

- `compute_eer` sweeps every observed threshold to find where False Accept Rate $\approx$ False Reject Rate — the Equal Error Rate.
- Reports accuracy at that threshold, plus the full ROC curve and its area (AUC).

Real outcome: EER = 0.010 (1%), Accuracy@EER = 0.990, AUC = 0.999 — a very strong verification result.

2.9 Visual proof

Two saved PNGs give concrete, inspectable evidence beyond the summary numbers: (1) a genuine pair and an impostor pair, each scored against the EER threshold with a MATCH / NO MATCH verdict overlay; (2) a gallery-matching demo — one embedding enrolled per identity, then 5 random query images matched against the whole gallery by nearest cosine similarity, each labeled correct or incorrect.

3. Computer Vision Techniques Used for Face

3.1 Detection — MTCNN

MTCNN (Multi-task Cascaded Convolutional Networks; Zhang et al., 2016) is a three-stage cascade of small CNNs used purely as the detection/localization front-end, not as the identity-recognition model itself:

- **P-Net (Proposal Net)** — scans the image at multiple scales, cheaply proposes many candidate face bounding boxes.
- **R-Net (Refine Net)** — rejects most candidates as false positives, refines the surviving boxes.
- **O-Net (Output Net)** — final refinement, plus regresses 5 facial landmarks: left eye, right eye, nose tip, left mouth corner, right mouth corner.

The cascade structure means most of the image is discarded cheaply by P-Net before the more expensive R-Net/O-Net stages run on the survivors, which is why MTCNN is fast enough to use as a preprocessing step.

```
detector = MTCNN(image_size=FACE_INPUT_SIZE, margin=0,
                  post_process=True, device=self.device)
aligned = detector(pil_image)
```

3.2 Alignment — classical geometry, not learned

Given the 5 landmarks, facenet-pytorch's `MTCNN.__call__` computes an **affine transform** that warps the detected face so the landmarks land at fixed canonical positions, then crops to 160×160. This step has zero learned parameters — it is deterministic geometric image processing (an affine transform solved from point correspondences).

Why this matters: a recognition network trained on canonically-posed faces performs substantially worse on faces at arbitrary rotation/scale/position. Alignment removes that variance *before* the embedding network has to learn to be invariant to it — more accurate, and requires far less training data than expecting the network to learn pose invariance itself.

The output is renormalized back to a standard uint8 RGB array so the rest of the pipeline never has to know facenet-pytorch's internal tensor normalization convention:

```
arr = ((arr * 128.0) + 127.5).clip(0, 255).astype(np.uint8)
```

4. InceptionResnetV1 Architecture, in Detail

InceptionResnetV1 (Szegedy et al., "*Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning*," 2016) combines two ideas that were separately state-of-the-art:

- **Inception modules** — instead of one filter size per layer, run several convolutions of different receptive fields (1×1 , 3×3 , stacked- 3×3 approximating 5×5) *in parallel* on the same input and concatenate their outputs. Captures both fine detail and broader spatial context at the same depth.
- **Residual connections** — each block's output is added back to its input ($x + F(x)$) rather than replacing it. This is what makes very deep networks trainable without vanishing gradients, and is why Inception-ResNet converges faster and more reliably than plain Inception at the same depth.

The concrete stack, as implemented by facenet-pytorch:

Stage	Role
Stem	A handful of stride-2 convolutions that quickly downsample the 160×160 input while extracting low-level edge/color/texture features.
5× Inception-ResNet-A	Operate on the largest feature-map grid; parallel $1\times1/3\times3$ branches + residual add.
Reduction-A	Stride-2 block that halves spatial resolution.
10× Inception-ResNet-B	Mid-level feature composition (edges → facial-part shapes: eyes, nose, mouth).
Reduction-B	Another downsampling stage.
5× Inception-ResNet-C ("block8")	Highest-level, most abstract feature composition, on the smallest, deepest feature map. This is the stage unfrozen during fine-tuning.
Global avg pool → dropout → last_linear	Fully-connected "bottleneck" layer projecting pooled features down to the 512-d embedding.
last_bn	BatchNorm applied to the final 512-d embedding vector.

This is exactly why freezing everything except `block8`, `last_linear`, and `last_bn` makes sense: the stem through Reduction-B extract generic low/mid-level facial structure that transfers well as-is, while only the final Inception-ResNet-C stack and the bottleneck projection — the part responsible for turning high-level facial features into a specific embedding coordinate — need to adapt to this project's own identities.

5. VGGFace2 Pretraining and Transfer Learning

VGGFace2 (Cao et al., 2018) is a dataset of approximately **3.31 million face images across ~9,131 identities**, deliberately curated for wide variation in pose, age, ethnicity, and lighting per identity — far larger and more diverse than any single downstream project could realistically collect and label.

facenet-pytorch trained InceptionResnetV1 on VGGFace2 as an identity-classification problem, and it is that pretrained `state_dict` — every conv filter, every BatchNorm statistic, through the entire stack in Section 4 — that gets loaded by:

```
backbone = InceptionResnetV1(pretrained='vggface2', classify=False).to(DEVICE)
```

`classify=False` matters: it tells the module to return the 512-d bottleneck embedding from `forward()` instead of the original VGGFace2 classification logits, since the classification head itself is discarded — this project only ever wants the embedding.

This is **transfer learning**: the enormous VGGFace2 pretraining pass already solved "what visual features distinguish faces" in general; fine-tuning only has to solve the much smaller problem of adapting the last stage of that feature space to this project's own enrolled identities, using a tiny dataset (2,065 training images) that would be far too small to train a network this deep from scratch.

6. ArcFace: the Shared Fine-Tuning Loss

Both the face and fingerprint notebooks fine-tune with the same loss function: **ArcFace** (Additive Angular Margin Loss; Deng, Guo, Xue & Zafeiriou, CVPR 2019), implemented in `models/common/arcface.py`.

A network trained with plain softmax cross-entropy learns features separable enough to classify the specific *training* identities — but biometric verification needs something stronger: at test time the system compares two embeddings of people it may never have seen labeled together, and decides "same person or different" via a similarity threshold. That requires **large angular margins between different identities and tight clustering within the same identity**, not just enough separability to pick the right softmax class.

```
cosine = F.linear(F.normalize(embeddings), F.normalize(self.weight))
sine   = torch.sqrt((1.0 - cosine.pow(2)).clamp(min=0, max=1))
phi    = cosine * self.cos_m - sine * self.sin_m          # cos(theta + m)
phi    = torch.where(cosine > self.threshold, phi, cosine - self.mm)

one_hot = torch.zeros_like(cosine)
one_hot.scatter_(1, labels.view(-1, 1).long(), 1)
output = (one_hot * phi + (1.0 - one_hot) * cosine) * self.s
```

For the correct identity class, the model is penalized as if the angle between the embedding and that class's center were m radians ($m = 0.5$, roughly 28.6°) larger than it actually is — forcing the network to push same-identity embeddings closer together and different-identity embeddings farther apart than plain softmax would ever require. The scale factor $s = 30.0$ rescales cosine values before the softmax so gradients remain well-behaved (raw cosines are confined to $[-1, 1]$, which produces a very "flat" softmax otherwise). The `torch.where` guard exists because $\cos(\theta+m)$ is not monotonic for angles near π , so beyond a threshold angle the code falls back to a linear penalty instead — a standard numerical-stability fix from the original paper.

ArcFace is training-only scaffolding — at inference time only the backbone's raw embedding is used; the ArcFace head's class-center weights are discarded once the checkpoint is saved. Using one shared loss across both modalities also means their results are directly comparable: when fingerprint's numbers came back much weaker than face's, the difference can be attributed to data/capacity factors, not to a fundamentally different and less effective training approach for that modality.

7. Face — Real Training Outcome, Explained

The strong EER/AUC numbers (EER 1%, AUC 0.999) come from three factors stacking together:

- **VGGFace2 pretraining** already gives a very good starting embedding space — the network has seen ~3.31M faces across ~9,131 identities before ever seeing this project's data.
- **ArcFace's margin loss** sharpens class separation further during the brief fine-tune, directly optimizing for the angular-margin property verification needs.
- **LFW's identities are visually distinct** (different people, controlled-ish web-photo conditions), so verification is genuinely easier here than on harder unconstrained or adversarial benchmarks.

The one operational wrinkle: this run trained on **CPU** due to the CUDA smoke-test fallback described in Section 2.1 — it still converged cleanly in 10 epochs, just slower than a GPU run would have been.

8. Fingerprint Training Notebook — Step by Step

8.1 Setup

Identical pattern to the face notebook: clones the repo at `phase-1-foundation`, installs only `h5py` (no `facenet-pytorch` needed here), and runs the same CUDA smoke test with CPU fallback. In the logged run this again fell back to CPU.

8.2 Dataset

Uses **SOCOFing** (Sokoto Coventry Fingerprint Dataset), attached natively via Kaggle's `dataset_sources` — no manual download. Only the **Real** (unaltered) split is used; SOCOFing's Altered/Obliterated splits simulate damaged prints and are out of scope for Phase 1. Filenames encode the subject id as a prefix (e.g. `1__M_Left_index_finger.BMP`), extracted with a regex and remapped to dense integer labels.

Real run: **6,000 images across 600 subjects** — 10 images per subject, a much thinner per-identity sample than face's ~49 images per identity, which becomes important in Section 12.

8.3 Preprocess (enhance + normalize) every image

Every image goes through `preprocessing/fingerprint.py`'s `FingerprintPreprocessor` — entirely classical CV, detailed in Section 9. Real run: all 6,000 images enhanced successfully (no detection step, so nothing is dropped, unlike face's MTCNN stage).

8.4 Train / validation / test split

Same per-identity 70/15/15 split logic as face. Real run: **train = 4,200, val = 600, test = 1,200** — exactly 7/1/2 images per subject, since every subject has exactly 10.

8.5 Model: ResNet50 + projection head + ArcFace

`FingerprintEmbeddingNet` (Section 10) with an `ArcMarginProduct` head sized to 600 classes. Only `layer4` plus the projection head are unfrozen:

```
for name, param in model.backbone.named_parameters():
    param.requires_grad = name.startswith('7') # '7' == layer4 index
```

8.6 Training loop

Pixel normalization here is `/255.0` then `(x-0.5)/0.5` (the standard ResNet/ImageNet-style `[-1, 1]` convention — distinct from face's `(x-127.5)/128`). Same loop structure as face: Adam (`lr=1e-4`), batch size 32, but **12 epochs**.

Epoch	Train Loss	Train Acc	Val Acc
1	21.5573	0.000	0.000
2	20.4074	0.000	0.000
3	18.6129	0.000	0.000

4	15.3910	0.000	0.000
5	12.3353	0.001	0.000
6	10.0376	0.011	0.000
7	7.8937	0.050	0.000
8	6.0893	0.185	0.000
9	4.7220	0.338	0.000
10	3.7296	0.495	0.000
11	2.9986	0.610	0.000
12	2.5078	0.682	0.000

Train accuracy climbs to 68% by epoch 12 — but **validation accuracy stayed at exactly 0.000 for all 12 epochs**. This is the critical signal, unpacked fully in Section 12.

8.7 Save the checkpoint

Same pattern as face: only `model.state_dict()` (backbone + projection, not the ArcFace head) is saved to `fingerprint_embedder.pt` and `.h5`.

8.8 Evaluation — Experiment 1

Same `run_modality_experiment` pipeline as face: reload through the production `FingerprintEmbedder`, extract embeddings for the 1,200 test images, build genuine/impostor cosine-similarity pairs, compute EER/AUC.

Real outcome: EER = 0.445, Accuracy@EER = 0.555, AUC = 0.579.

8.9 Visual proof

Same genuine-vs-impostor pair visualization pattern as face, saved to `evaluation/results/fingerprint_pair_test.png`.

9. Computer Vision Techniques Used for Fingerprint

Fingerprint preprocessing (`preprocessing/fingerprint.py`) is **entirely classical image processing** — no learned weights anywhere in this stage — three sequential steps addressing three different sources of noise in a raw scan.

9.1 CLAHE — Contrast Limited Adaptive Histogram Equalization

```
self._clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
contrast_enhanced = self._clahe.apply(gray)
```

Standard global histogram equalization redistributes the whole image's intensity histogram to use the full dynamic range — but a fingerprint scan often has locally varying contrast (drier vs. oilier regions of the same finger, uneven scanner pressure), which a single global transform can't fix and can even over-amplify noise in already-low-contrast regions. CLAHE instead equalizes contrast **locally**, within small 8×8 tiles, and clips the amplification factor (`clipLimit=2.0`) to avoid over-amplifying noise in near-uniform tiles.

9.2 Statistical ridge normalization

```
normalized = target_mean + np.sign(gray - mean) * \
    np.sqrt(target_var * (gray - mean) ** 2 / var)
```

Rescales every pixel so the image has a fixed mean (100.0) and variance (100.0), preserving the *sign* of each pixel's deviation from the mean — ridges stay darker than valleys, valleys stay lighter than ridges — while standardizing overall brightness/contrast. This is the standard normalization step used before ridge-orientation-based enhancement in the fingerprint literature (e.g. Hong, Wan & Jain), and exists so that two scans of the same finger captured with different scanner pressure or calibration land on a comparable intensity scale.

9.3 Gabor filter bank enhancement

```
for theta in np.arange(0, np.pi, np.pi / 8): # 8 orientations
    kernel = cv2.getGaborKernel((15, 15), sigma=4.0, theta=theta,
                                lambda=10.0, gamma=0.5, psi=0)
    filtered = cv2.filter2D(gray, cv2.CV_64F, kernel)
    enhanced = np.maximum(enhanced, filtered)
```

A Gabor filter is a sinusoidal plane wave modulated by a Gaussian envelope — it responds most strongly to texture oriented at a specific angle (θ) and spatial frequency (λ). Fingerprint ridges are locally near-parallel oriented texture, exactly what Gabor filters are the standard tool for enhancing in the fingerprint literature. The code applies a **bank of 8 filters** spanning 0°-157.5° in 22.5° steps, and takes the pixel-wise maximum response across all 8 — so at every pixel, whichever filter's orientation best matches the local ridge direction dominates the output, sharpening ridge/valley contrast regardless of which direction ridges run in that region (ridge orientation varies continuously across a real fingerprint, especially near the core and deltas).

The final enhanced image is resized to 224×224 (matching ResNet50's expected input) and replicated to 3 channels.

10. ResNet50 + Projection Head Embedding

`models/fingerprint/inference.py`'s `FingerprintEmbeddingNet` is an ImageNet-pretrained **ResNet50** (IMAGENET1K_V2 weights) with its 1,000-class classification head stripped off, followed by a trainable `Linear(2048 → 256)` projection to the final 256-d embedding:

```
backbone = tv_models.resnet50(weights=tv_models.ResNet50_Weights.IMAGENET1K_V2)
self.backbone = nn.Sequential(*list(backbone.children())[:-1]) # drop FC head
self.projection = nn.Linear(2048, FINGERPRINT_EMBEDDING_DIM) # 2048 -> 256
```

This is explicitly documented as a **DeepPrint substitute**. DeepPrint is the fingerprint-specific architecture from the research literature, but it has no public pretrained weights or pip-installable reference implementation, making it impractical to reproduce reliably within a capstone timeline. A ResNet50 backbone fine-tuned with an ArcFace-style angular-margin loss is a well-documented, reproducible alternative that still yields a fixed-length, discriminative fingerprint embedding.

Unlike face's InceptionResnetV1 (pretrained specifically *for face verification* on VGGFace2), ResNet50's pretraining is **generic ImageNet classification** (distinguishing 1,000 everyday object categories like cats, cars, and chairs) — a different domain entirely from fingerprint ridge texture. This is why only Layer 4 (the final residual block) and the projection head are unfrozen: the low-level filters (edges, blobs, textures) transfer reasonably well as generic feature detectors, but the network starts from a much weaker prior for this specific task than face's did.

11. Fingerprint — Real Training Outcome

EER = 0.445, Accuracy@EER = 0.555, AUC = 0.579. Read plainly: an AUC of 0.579 is barely better than random guessing (0.5 = no discriminative power at all), and an EER of 44.5% means the system is wrong on nearly half of all decisions at its best possible threshold — close to useless as a real verification threshold, despite training accuracy reaching 68%.

This gap between a decent-looking training accuracy and a near-random verification result is the single most important thing to understand about this run, and it is unpacked in full in Section 12.

12. Why Fingerprint Underperformed Face — Honest Analysis

The project's own report (docs/PROJECT_REPORT.md, Section 8.2) flags this explicitly rather than hiding it. The root cause is textbook **overfitting relative to class count versus unfrozen capacity**:

- **600 identity classes** with only **7 training images per class**, versus face's 62 classes with far more images per identity (2,065 training images / 62 = ~33 per identity).
- Only layer4 + the projection head are trainable — a small amount of adaptable capacity being asked to separate 600 classes starting from a generic ImageNet prior (features learned for "cat vs. dog vs. car," not for fingerprint ridge discrimination) is a much harder ask than fine-tuning VGGFace2 weights already trained for face verification specifically.
- The result: the network **memorized the specific training images** (train accuracy rising steadily to 68%) rather than learning ridge-pattern features that generalize to unseen images of the same finger — exactly what a validation accuracy frozen at 0.000 for all 12 epochs means. It never once correctly classified a held-out image, even as it got better and better at the training set.

Factor	Face (worked well)	Fingerprint (failed to generalize)
Identity classes	62	600
Images per identity (train)	~33	7
Backbone pretraining domain	Face verification (VGGFace2)	Generic ImageNet classification
Unfrozen capacity	Last Inception-ResNet-C block + bottleneck	Last ResNet block (layer4) + projection
Val accuracy trend	Climbed steadily to 0.913	Stuck at 0.000 the entire run
Final AUC	0.999	0.579 (barely above chance)

The documented fix path (not yet applied in Phase 1): more training epochs at a lower learning rate, unfreezing more of the backbone with stronger regularization (dropout, weight decay), or evaluating against held-out identities never seen during training rather than a random image-level split. This is presented in the project documentation as a known, explained limitation, not a silently accepted bad number.

13. Side-by-Side Comparison

	Face	Fingerprint
Dataset	LFW (via sklearn)	SOCOFing (Kaggle-native)
Raw samples	3,023 images / 62 identities	6,000 images / 600 subjects
After preprocessing	2,984 (39 dropped, no face found)	6,000 (no drops)
Train / Val / Test	2,065 / 423 / 496	4,200 / 600 / 1,200
Preprocessing	MTCNN detection + affine alignment	CLAHE + ridge normalization + Gabor bank
Backbone	InceptionResnetV1	ResNet50
Backbone pretraining	VGGFace2 (face-specific, ~9,131 IDs)	ImageNet (generic objects, 1,000 classes)
Embedding dimension	512	256
Unfrozen layers	block8 + last_linear + last_bn	layer4 + projection head
Fine-tuning loss	ArcFace (m=0.5, s=30)	ArcFace (m=0.5, s=30)
Epochs	10	12
Device actually used	CPU (GPU smoke test failed)	CPU (GPU smoke test failed)
Final val accuracy	0.913	0.000
EER	0.010	0.445
Accuracy @ EER threshold	0.990	0.555
ROC-AUC	0.999	0.579
Verdict	Strong, production-viable result	Overfit — needs the fixes in Section 12

Generated from `kaggle_kernels/face_training/face-embedding-training.ipynb`,
`kaggle_kernels/fingerprint_training/fingerprint-embedding-training.ipynb`, their Kaggle run logs (`kaggle_output/face/` and
`kaggle_output/fingerprint/`), and `docs/COMPUTER_VISION.md` / `docs/PROJECT_REPORT.md`.